University of Engineering and Technology, Lahore

Lab_7_Report

Submitted by: 2022_MC_27

1. Objective

This lab builds upon Lab Manual 5 by enabling autonomous navigation using a pre-built map, the Nav2 navigation stack, and the TurtleBot3 platform inside a Gazebo simulation. The key objectives are:

- Load a previously saved occupancy map (my_map.pgm / my_map.yaml) into Nav2.
- Localize the robot on the map using AMCL (Adaptive Monte Carlo Localization).
- Send single navigation goals via RViz and the command line.
- Plan and execute multi-waypoint missions using a custom ROS 2 Python node.
- Observe and interpret the global costmap, local costmap, and AMCL particle cloud.
- Understand Nav2 recovery behaviors when obstacles block the planned path.

2. Introduction

Nav2 (Navigation 2) is the standard autonomous navigation framework for ROS 2. It coordinates path planning, trajectory following, localization, and failure recovery through a set of interconnected servers. This lab uses Nav2 together with a pre-built map and the TurtleBot3 Burger robot to demonstrate full autonomous navigation in a Gazebo simulation environment. The key components of the Nav2 stack include the BT Navigator, which executes behavior trees to orchestrate the entire navigation task; the Planner Server, which computes a global path from the current pose to the goal; the Controller Server, which follows the global path using a local trajectory controller; the Map Server, which loads and publishes the static occupancy grid; and the Recovery Behavior Server, which handles failure cases through spin-in-place, back-up, and wait behaviors.

Localization is handled by AMCL (Adaptive Monte Carlo Localization), which represents the robot's position estimate as a probability distribution modeled by a set of weighted particles. Each particle is a candidate pose (x, y, yaw). When new LiDAR scan data arrive, particles consistent with the scan receive higher weights and are more likely to survive resampling. Over time the particle cloud converges to the true robot pose. A good initial pose estimate, set using the 2D Pose Estimate tool in RViz, is critical because a poor starting estimate causes the particle cloud to diverge and results in incorrect path planning.

The map used in this lab was produced in Lab 5 using Cartographer SLAM and consists of two files: my_map.pgm, a grayscale image where white pixels represent free space, black pixels represent obstacles, and grey pixels represent unknown regions; and my_map.yaml, a metadata file specifying the map resolution in meters per pixel, the origin coordinates, and the occupancy thresholds used to interpret pixel values. Together these files provide Nav2 with the complete static environment model needed for global path planning and localization.

3. Environment Setup and Launch Procedure

3.1 Environment Variables

The following commands must be sourced in every terminal (or added to ~/.bashrc):

```
source /opt/ros/humble/setup.bash
export TURTLEBOT3_MODEL=burger
```

3.2 Step 1 – Launch Gazebo Simulation

Launch the same TurtleBot3 world used in Lab 5 so the robot starts at the same origin as the saved map:

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

3.3 Step 2 – Launch Nav2 with Saved Map

Open a new terminal and launch Nav2 bringup pointing to the saved map:

```
ros2 launch turtlebot3_navigation2 navigation2.launch.py \
    use_sim_time:=True \
    map:=$HOME/maps/my_map.yaml
```

This command starts the Map Server, AMCL, Planner Server, Controller Server, BT Navigator, and opens a pre-configured RViz window showing the Nav2 visualization panels.

3.4 Step 3 – Set Initial Pose Estimate in RViz

Because AMCL begins with a uniformly spread particle cloud, an initial pose estimate must be provided:

- In RViz, click the 2D Pose Estimate button in the toolbar.
- Click the map at the robot's approximate starting location (near the origin).
- Drag the arrow in the direction the robot is facing (positive X axis).
- Observe the AMCL particle cloud (red arrows) converging around the estimate.
- Drive the robot slightly using teleoperation to help AMCL refine localization:

```
ros2 run turtlebot3_teleop teleop_keyboard
```

- Confirm that the LiDAR scan overlay in RViz aligns with the map walls.

4. Tasks and Observations

Task 1: Single Goal Navigation

Procedure

Three separate single navigation goals were sent using the RViz Nav2 Goal button. For each goal the start pose was noted by echoing the /amcl_pose topic, the goal was clicked on the map, and the result was observed.

```
ros2 topic echo /amcl_pose
```

Goals could also be sent programmatically from the command line:

```
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
"{pose: {header: {frame_id: 'map'}, pose: {position: {x: 1.0, y: 0.5, z: 0.0},
    orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}}}}"
```

Observations – Goal 1

Start Pose (Before Navigation)	End Pose (After Navigation)
x = -0.048	x = 1.76
y = -0.053	y = -1.39
z = 0.0	z = 0.0
orientation z = -0.001	orientation z = -0.025
orientation w = 0.999	orientation w = 0.9996

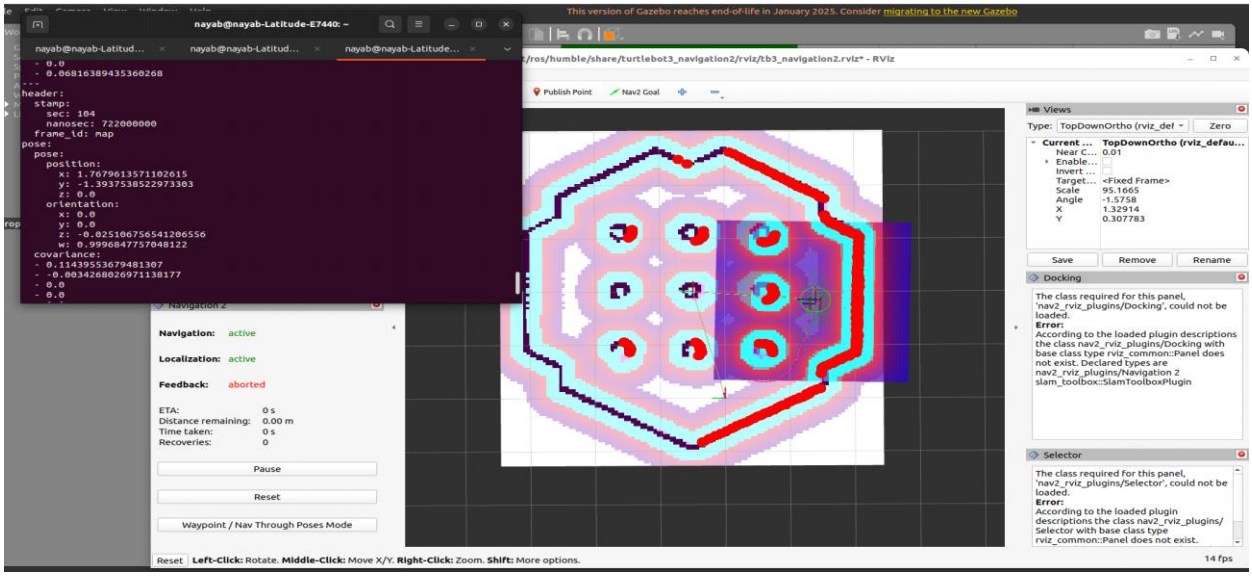

Figure 1: Robot navigating to Goal 1; global path shown in blue

Observations – Goal 2

Start Pose (Before Navigation)	End Pose (After Navigation)
x = 1.76	x = 2.26
y = -1.39	y = 2.59
z = 0.0	z = 0.0
orientation z = -0.025	orientation z = 0.80
orientation w = 0.9996	orientation w = 0.59

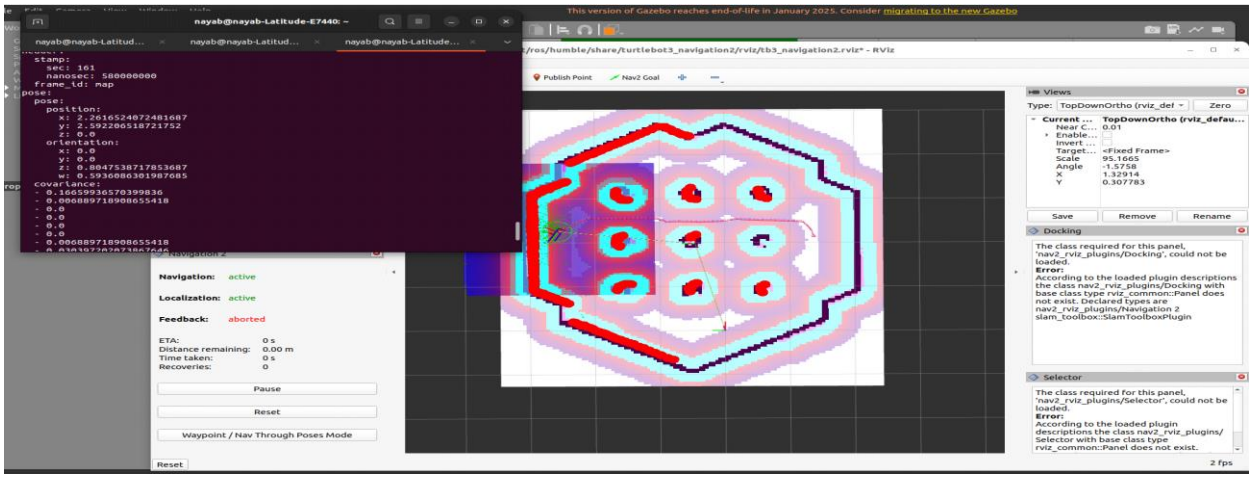

Figure 2: Robot navigating to Goal 2

Observations – Goal 3

Start Pose (Before Navigation)	End Pose (After Navigation)
x = 2.26	x = 3.70
y = 2.59	y = 1.18
z = 0.0	z = 0.0
orientation z = 0.80	orientation z = -0.65
orientation w = 0.59	orientation w = 0.758

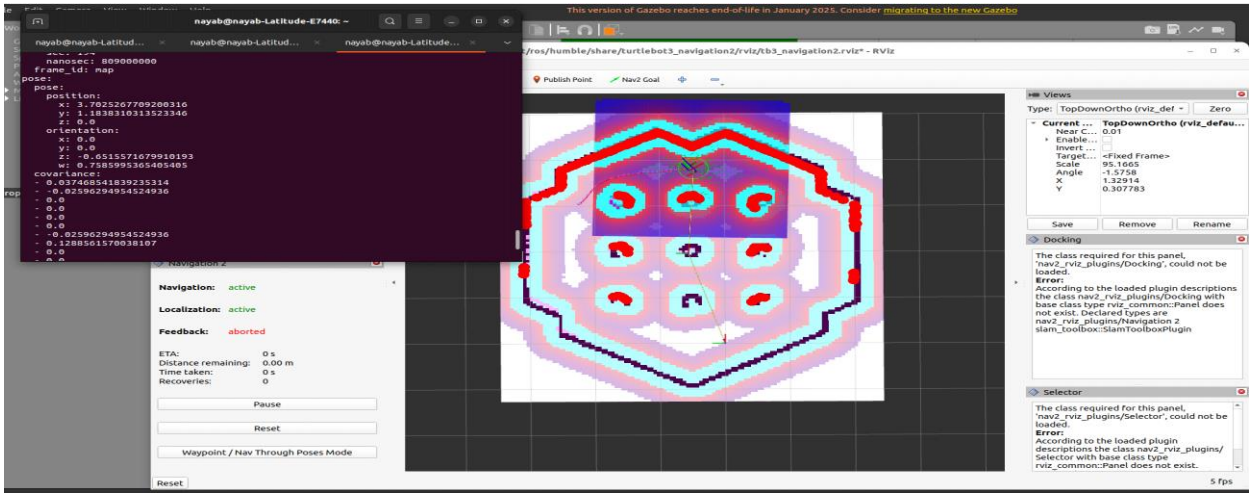

Figure 3: Robot navigating to Goal 3

Task 2: Multi-Waypoint Mission

4.2.1 Waypoint Definition

The following five waypoints were identified from the RViz coordinate grid and confirmed with ros2 topic echo /amcl_pose. All waypoints lie in free space on the saved map.

WP #	X	Y	Z	Yaw	Description
1	0.535	-0.035	0	-0.004	Move forward from origin
2	1.295	0.451	0	1.661	Left area of map
3	1.006	-0.256	0	-1.013	Right area of map
4	0.04	-0.456	0	0.197	Bottom-right area of map
5	0.05	0.031	0	0.291	Return to origin

Table 1: Five waypoints used for the multi-waypoint mission

4.2.2 Code Explanation – pose_printer.py (used during Task 2 to record poses)

The pose_printer.py node was written to read real-time robot pose from the /amcl_pose topic and print it in a clean, human-readable format. This was particularly useful during Task 2 to record exact waypoint coordinates while manually driving the robot to each position.

```
# pose_printer.py - subscribes to /amcl_pose and prints X, Y, Z, Yaw

import rclpy
from rclpy.node import Node
from geometry_msgs.msg import PoseWithCovarianceStamped
import math

class PosePrinter(Node):
    def __init__(self):
        super().__init__('pose_printer')
        self.sub = self.create_subscription(
            PoseWithCovarianceStamped, '/amcl_pose', self.callback, 10)

    def quat_to_yaw(self, x, y, z, w):
        # Convert quaternion to yaw angle using atan2
        siny_cosp = 2.0 * (w * z + x * y)
        cosy_cosp = 1.0 - 2.0 * (y * y + z * z)
        return math.atan2(siny_cosp, cosy_cosp)

    def callback(self, msg):
        x   = msg.pose.pose.position.x
        y   = msg.pose.pose.position.y
        z   = msg.pose.pose.position.z
        q   = msg.pose.pose.orientation
        yaw = self.quat_to_yaw(q.x, q.y, q.z, q.w)
        print(f'X: {x:.3f} | Y: {y:.3f} | Z: {z:.3f} | Yaw: {yaw:.3f}')
```

Key points: The callback fires every time AMCL publishes an updated pose estimate. Because AMCL uses a quaternion for orientation (x, y, z, w), the helper quat_to_yaw() converts it to a human-readable yaw angle in radians using the standard atan2 formula. Position z is always 0.0 for a ground robot.

Run alongside Nav2 with:

```
python3 pose_printer.py
```

4.2.3 Code Explanation – task_2.py (FollowWaypoints mission)

task_2.py uses the Nav2 FollowWaypoints action server to send all five waypoints as a single batch goal. The action server plans a path through all waypoints sequentially without requiring a separate goal request per waypoint.

```
# Key design decisions in task_2.py

# 1. Action client targets 'follow_waypoints'
self._client = ActionClient(self, FollowWaypoints, 'follow_waypoints')

# 2. make_pose() builds a PoseStamped for each waypoint
#    position.z is always 0.0 (ground robot)
#    orientation uses z and w quaternion components only
def make_pose(x, y, z_orient, w_orient):
    pose = PoseStamped()
    pose.header.frame_id = 'map'
    pose.pose.position.x = x
    pose.pose.position.y = y
    pose.pose.position.z = 0.0          # always 0 for TurtleBot3
    pose.pose.orientation.z = z_orient  # controls heading
    pose.pose.orientation.w = w_orient  # controls heading
    return pose

# 3. feedback_callback() prints arrival at each waypoint
#    using current_waypoint index from feedback
def feedback_callback(self, feedback_msg):
    current_index = feedback_msg.feedback.current_waypoint
    if current_index != self._last_printed_waypoint:
        ...  # log reached/upcoming waypoint info

# 4. All 5 poses are sent as a list in one goal message
goal_msg = FollowWaypoints.Goal()
goal_msg.poses = waypoints
self._client.send_goal_async(goal_msg, feedback_callback=...)
```

Orientation reference used in task_2.py:

```
# Facing forward  (+X): z_orient=0.0,  w_orient=1.0
# Facing left     (+Y): z_orient=0.7,  w_orient=0.7
# Facing backward (-X): z_orient=1.0,  w_orient=0.0
# Facing right    (-Y): z_orient=-0.7, w_orient=0.7
```

Limitation: FollowWaypoints does not always generate a fresh global path per waypoint; it may string waypoints together, occasionally producing suboptimal trajectories near obstacles.

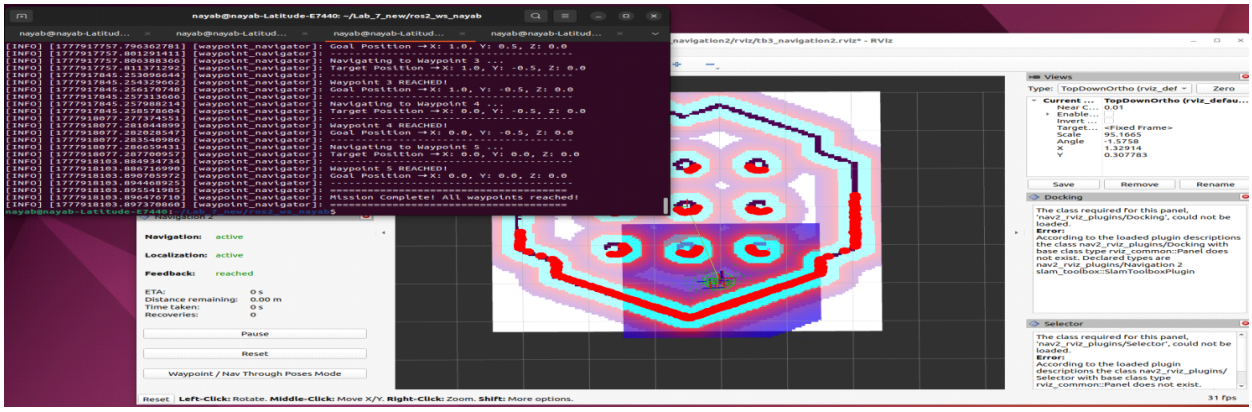

Figure 4: Multi-waypoint mission in progress – global path shown connecting all 5 waypoints

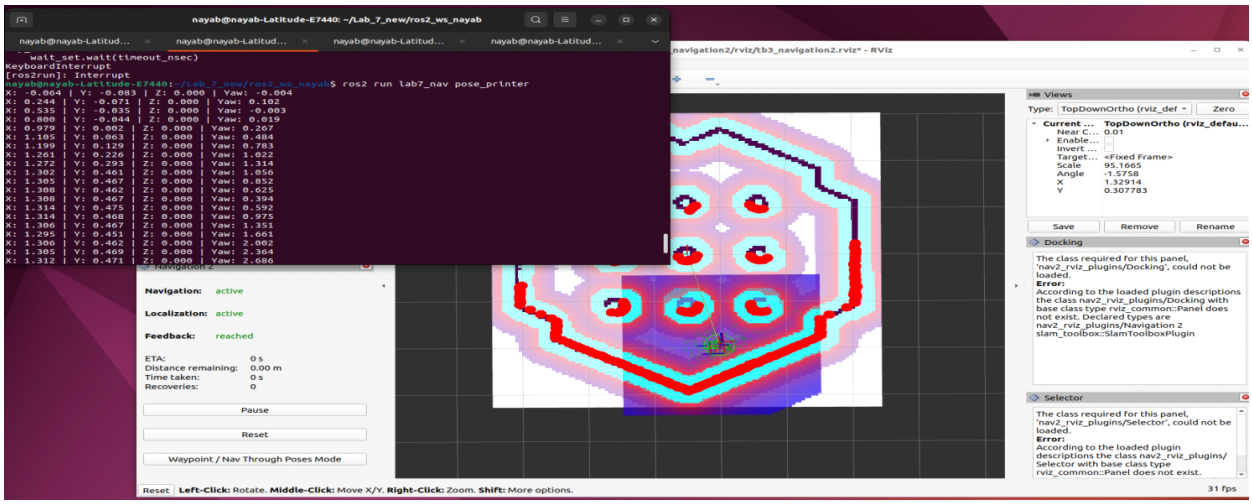

Figure 5: Terminal log confirming each waypoint reached during task_2.py execution

Task 3: Dynamic Waypoint Injection

4.3.1 Overview

task_3.py extends the waypoint navigator to accept waypoints at runtime either from command-line arguments or from a ROS 2 parameter. This eliminates the need to edit and re-run the Python file each time a new mission is required.

4.3.2 Code Explanation – task_3.py

The key difference from task_2.py is the use of NavigateToPose (one goal per waypoint) instead of FollowWaypoints (all at once). This ensures a fresh global path is planned for each waypoint, resulting in more accurate arrivals.

```
# task_3.py - Dynamic waypoint navigator using NavigateToPose

# Each waypoint gets its own NavigateToPose goal
self._client = ActionClient(self, NavigateToPose, 'navigate_to_pose')

# make_pose() now computes z automatically from w
def make_pose(x, y, w_orient, node):
    w_clamped = max(-1.0, min(1.0, w_orient))
```

```
    z_orient  = math.sqrt(1.0 - w_clamped ** 2)  # enforce unit quaternion
    ...

# Command-line parsing: groups of 3 numbers per waypoint
# python3 task_3.py 0.5 0.0 1.0  1.0 0.5 0.7  0.0 0.0 1.0
def _parse_argv_waypoints(node):
    raw = sys.argv[1:]
    values = [float(v) for v in raw]
    # Every 3 values = one waypoint (x, y, orientation_w)
    for i in range(0, len(values), 3):
        x, y, w = values[i], values[i+1], values[i+2]
        waypoints.append(make_pose(x, y, w, node))

# ROS 2 parameter alternative:
# ros2 run lab7_nav task_3 --ros-args \
#     -p "waypoints:=[0.5, 0.0, 1.0, 1.0, 0.5, 0.7, 0.0, 0.0, 1.0]"

# Status 4 = SUCCEEDED in ROS 2 goal status
if result.status == 4:
    self.get_logger().info(f'Waypoint {waypoint_number} REACHED!')
```

The navigator iterates through waypoints one at a time. After completing all waypoints it prints a mission summary listing any missed waypoints and their coordinates.

4.3.3 Example Usage

```
# Direct Python (3 waypoints: x y w for each)
python3 task_3.py 0.5 0.0 1.0  1.0 0.5 0.7  0.0 0.0 1.0

# Via ros2 run with parameter
ros2 run lab7_nav task_3 --ros-args \
    -p "waypoints:=[0.5, 0.0, 1.0, 1.0, 0.5, 0.7, 0.0, 0.0, 1.0]"
```

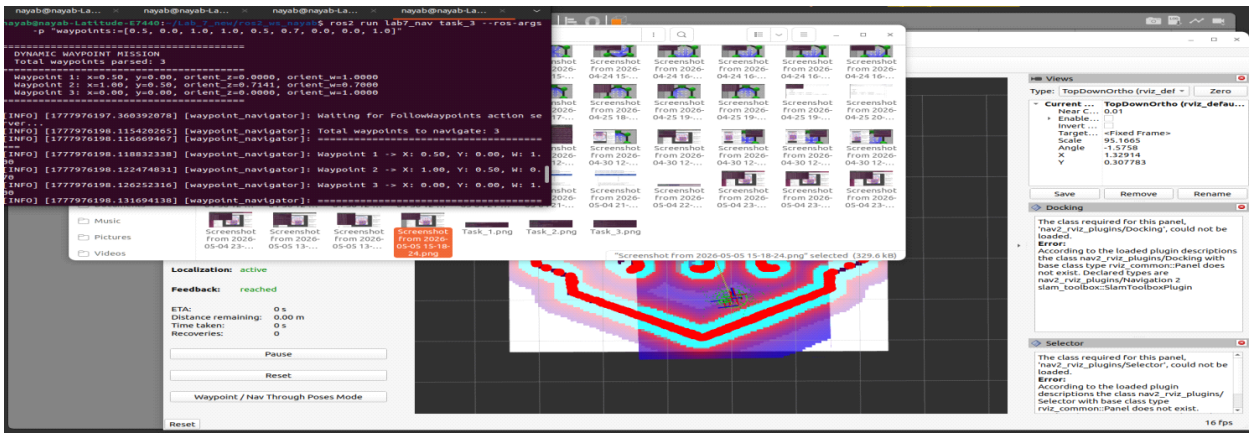

Figure 6: task_3.py receiving waypoints via command-line arguments and navigating dynamically

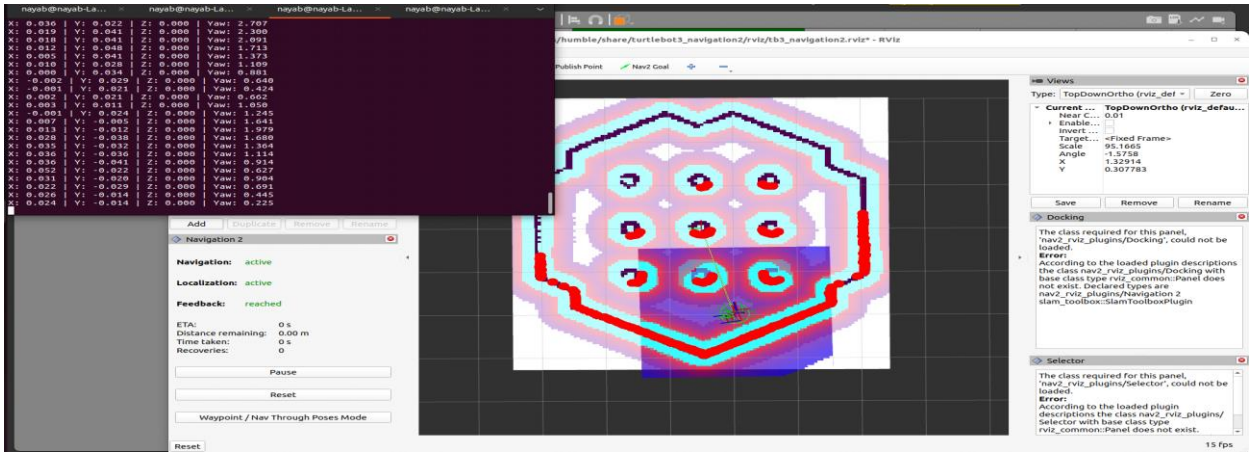

Figure 7: Individual red path segments replanned for each waypoint in task_3.py

Task 4: Costmap Observation

4.4.1 Procedure

The robot was navigated close to an obstacle using the Nav2 Goal button. The local and global costmaps were observed in RViz as the robot approached the wall.

4.4.2 Costmap Topics Identified

Costmap Type	RViz Topic	Role
Global Costmap	/global_costmap/costmap	Full-map inflation for global path planning. Updated infrequently.
Local Costmap	/local_costmap/costmap	Rolling window around the robot. Updated at high rate for real-time obstacle avoidance.
Global Path	/plan	Planned trajectory from planner server to the goal.
Local Path	/local_plan	Portion of path currently being followed by the DWB controller.

Table 2: Costmap topics and their roles

4.4.3 Observations

Global Costmap: The global costmap showed inflated obstacle regions (coloured rings) around each wall. These inflation layers prevent the global planner from routing the robot too close to obstacles, providing a safety margin.

Local Costmap: As the robot moved within ~0.5 m of a wall, the local costmap showed the inflation region in real time, causing the DWB controller to curve away from the obstacle and re-join the planned path at a safer distance.

The primary difference between the two: the global costmap covers the entire known map and is used once per goal to plan the initial path; the local costmap uses only a small rolling window centered on the robot and is updated continuously for reactive collision avoidance.

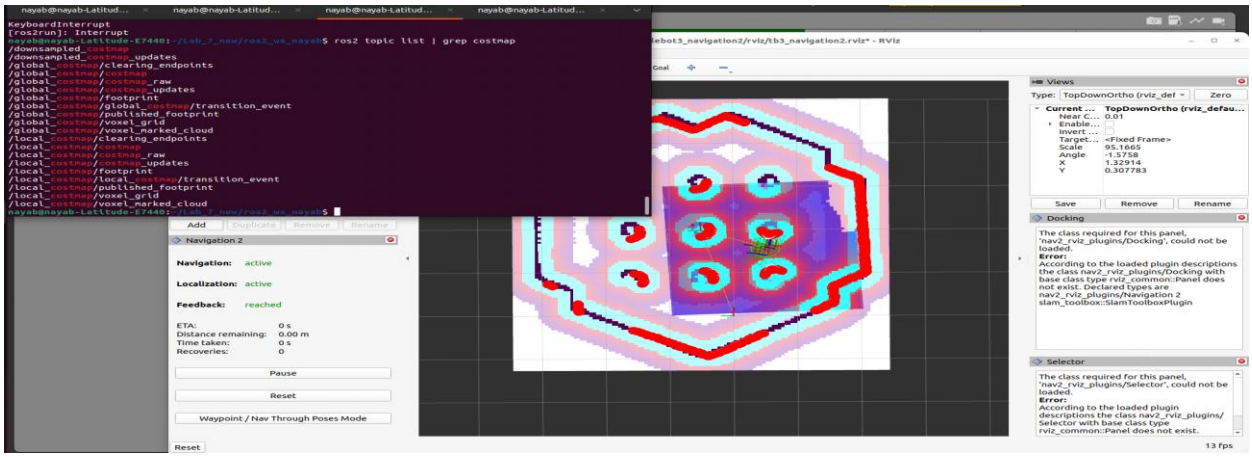

Figure 8: Global costmap (left panel) and local costmap (right) showing inflation near obstacles

Task 5: Navigation Recovery Behaviors

4.5.1 Procedure

An obstacle (a box model) was inserted into Gazebo directly in the robot's planned path. The Nav2 behavior server response was observed in RViz and in the terminal.

```
python3 task_5.py 0.5 0.0 1.0 1.0 0.5 1.0 1.0 -0.5 1.0
```

4.5.2 Code Explanation – task_5.py (Recovery Navigator)

task_5.py enhances task_3.py with an automatic retry/recovery loop. When a waypoint navigation attempt fails (status != SUCCEEDED), the node waits WAIT_BETWEEN_RETRIES seconds and resubmits the same goal, up to MAX_RECOVERY_ATTEMPTS times.

```
# task_5.py - Recovery behavior parameters
MAX_RECOVERY_ATTEMPTS = 5    # retry up to 5 times per waypoint
WAIT_BETWEEN_RETRIES    = 2.0  # seconds between retries
GOAL_TOLERANCE          = 0.05  # metres

# Recovery loop
for attempt in range(1, MAX_RECOVERY_ATTEMPTS + 1):
    ...send goal...
    if status == GoalStatus.STATUS_SUCCEEDED:
        return True
    if attempt < MAX_RECOVERY_ATTEMPTS:
        time.sleep(WAIT_BETWEEN_RETRIES)  # wait then retry

# Safe stop: cancel Nav2 goal + publish zero velocity
def stop_robot(self):
    self.goal_handle.cancel_goal_async()
    pub = self.create_publisher(Twist, '/cmd_vel', 10)
    stop_msg = Twist()  # all zeros = stop
    for _ in range(15): pub.publish(stop_msg)
```

4.5.3 Observed Recovery Behaviors

When the box was placed in the path, Nav2 exhibited the following sequence of behaviors:

- Replan: The Planner Server detected the new obstacle in the costmap and computed an alternative global path around the box (the red path line updated in RViz).
- Spin in place: When replanning was insufficient (box too close), the BT Navigator triggered a spin recovery behavior — the robot rotated 360° in place to update the LiDAR scan and the local costmap.
- Back-up: If spin did not resolve the situation, the robot backed up a short distance before attempting to replan again.
- Wait: As a last resort, the robot waited for a configurable timeout before declaring the goal failed.

4.5.4 Recovery Server Identification via rqt_graph

```
rqt_graph
```

In rqt_graph, **/behavior_server** is the sole recovery behavior server visible in the rqt_graph. It **responds** to recovery action calls from the BT Navigator nodes, executing spin, backup, and wait maneuvers as defined in the Nav2 behavior tree.

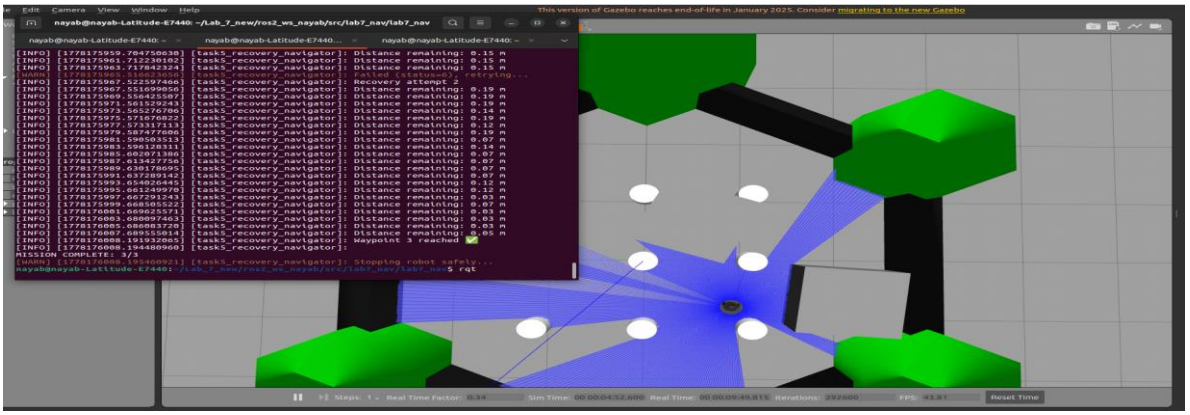

Figure 9: Box obstacle spawned in Gazebo blocking the robot's planned route

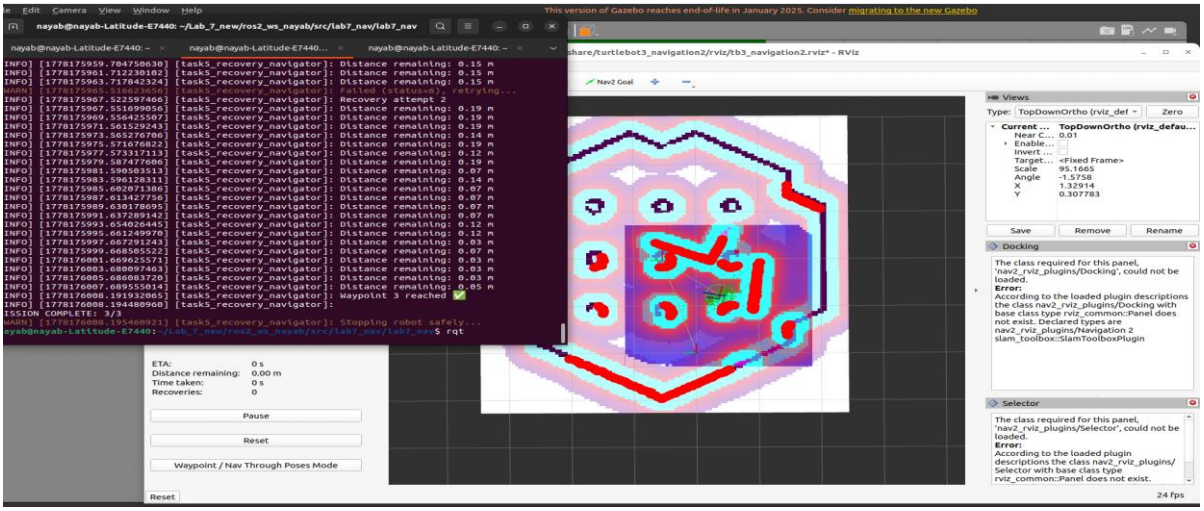

Figure 10: Nav2 replanning a new global path around the inserted box obstacle

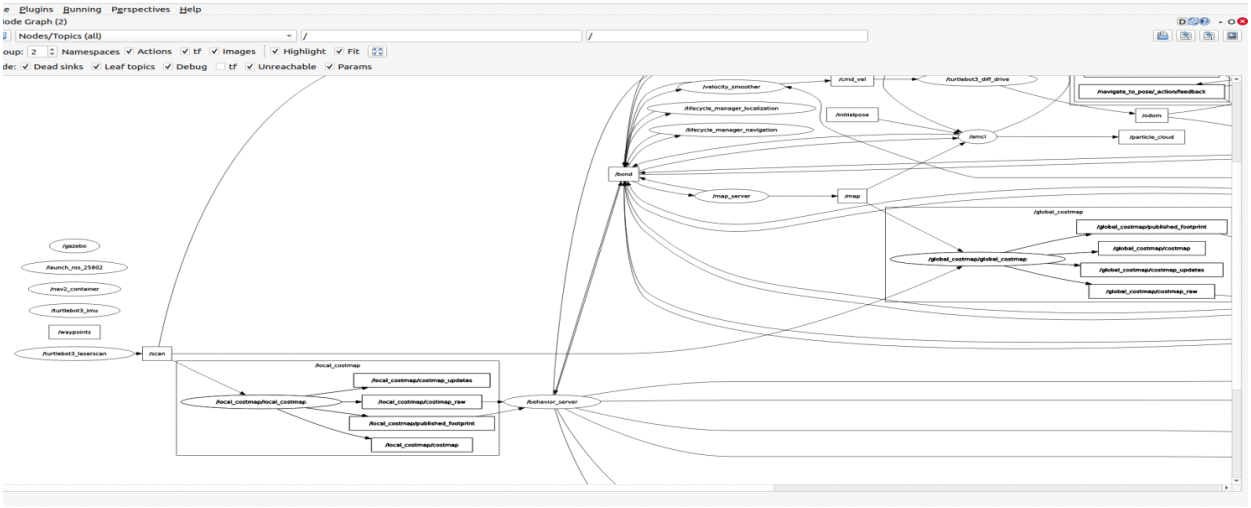

Figure 11: rqt_graph showing /behavior_server connected to the navigation action server

7. Conclusion

This lab successfully demonstrated the complete autonomous navigation pipeline in ROS 2 using the Nav2 stack. Starting from a pre-built occupancy map created in Lab 5, the TurtleBot3 Burger was localized on the map using AMCL, and a series of navigation tasks were executed with increasing complexity.

The key learning outcomes from this lab are:

- Nav2 integrates multiple specialized servers — planning, control, localization, recovery — into a cohesive navigation system governed by a behavior tree.
- AMCL provides reliable localization once a good initial pose estimate is provided; driving the robot briefly after setting the initial pose significantly improves convergence.
- FollowWaypoints is convenient for batch missions but offers less precise per-waypoint path control compared to individual NavigateToPose goals.
- Dynamic waypoint injection (task_3.py) eliminates the need to modify code between missions, enabling reusable, flexible navigation nodes.
- The global costmap and local costmap serve complementary roles: the global costmap enables obstacle-aware path planning, while the local costmap enables real-time reactive collision avoidance.
- Nav2's recovery behavior server (behavior_server) automatically handles blocked paths through a sequence of replan, spin, back-up, and wait behaviors, significantly improving mission robustness.

Comparison with Lab 5 (SLAM): Lab 5 focused on building a map in real time using Cartographer SLAM, where the robot had no prior knowledge of the environment. Lab 7 used that saved map as a static reference for localization and path planning. The two workflows are complementary: SLAM produces the map, and Nav2 consumes it for autonomous navigation. The transition from SLAM to navigation required understanding AMCL initialization, which is the critical bridge between the two workflows.

Several challenges were encountered during this lab. The most significant was setting the initial 2D Pose Estimate accurately in RViz — a slightly misaligned arrow caused the AMCL

particle cloud to diverge, making the robot plan paths relative to the wrong position on the map. This was resolved by driving the robot slowly with teleoperation after setting the estimate, which allowed AMCL to refine its localization using live LiDAR data. Another challenge was ensuring the robot spawned exactly at (0, 0, 0) in Gazebo; on one attempt a slight offset caused the laser scan to not align with the map walls, requiring a full Gazebo restart. During the multi-waypoint mission, Waypoint 3 (x=1.0, y=-0.5) was initially placed too close to a wall, causing the planner to reject the goal. Moving the waypoint slightly inward to open space resolved this issue. Finally, during Task 5, inserting the box obstacle too close to the robot caused Nav2 to immediately trigger recovery behaviors before even beginning to navigate, which required repositioning the obstacle further along the planned path to properly observe the replan and spin-in-place sequence.